Journal of Electrical Engineering & Technology (2026) 21:2099–2105
https://doi.org/10.1007/s42835-025-02541-7

ORIGINAL ARTICLE

Pipeline of Pruning, Knowledge Distillation, and Quantization for Model Compression

Woon Chae[1] · **Kisung Seo**[1]

Received: 18 July 2025 / Revised: 13 November 2025 / Accepted: 30 November 2025 / Published online: 19 December 2025
© The Author(s) under exclusive licence to The Korean Institute of Electrical Engineers 2025

Abstract
Model compression or lightweighting is a critical task in deep learning to reduce memory usage and improve processing speed. Pruning, quantization, and knowledge distillation have been individually studied as effective approaches. However, efforts to combine these techniques for synergistic performance improvements have been limited. In particular, due to data type changes introduced by quantization, it becomes difficult to apply pruning or distillation afterward, and most existing approaches are limited to determining only the order between pruning and distillation. In this paper, we first propose an integrated compression pipeline that effectively combines pruning, quantization, QAT (Quantization-Aware Training), and knowledge distillation, considering the trade-offs and synergistic efficiency among these processes. Second, we introduce an efficiency score defined as the harmonic mean between the model compression rate and accuracy drop, to provide a balanced evaluation between model size reduction and performance retention. Third, we propose a Compression-Aware Knowledge Distillation (CAKD) approach that eliminates the need for an external teacher model and employs a two-stage distillation process during both fine-tuning and QAT phases to achieve further performance improvement. We validate our method through experiments using the ResNet-56 and MobileNetV2 models on the CIFAR-100 and Tiny ImageNet datasets. The results show that our pipeline achieves a better trade-off between model compression and accuracy degradation compared to existing techniques.

Keywords Model compression · Pruning · Quantization · Knowledge distillation · Integrated pipeline

1 Introduction

Deep learning has demonstrated remarkable performance in computer vision and learning tasks, leading to its widespread adoption [1–4]. Meanwhile, the need for model compression has become critical for deploying deep models on embedded and mobile systems, where computational resources are limited [5]. Major approaches for model compression include pruning, knowledge distillation, and quantization.

Pruning [6] reduces model size and improves processing speed by removing less important or redundant filters across layers. Quantization [7] reduces memory usage and computation cost by converting model weights from 32-bit floating point (Float32) to lower-precision formats such as 16-bit floating point (FP16) or 8-bit integer (INT8). Knowledge Distillation (KD) [8] transfers knowledge from a large, pretrained teacher network to a smaller student network by minimizing the difference in output distributions. The goal is to train a compact network that mimics the performance of the larger one.

Previous research has primarily focused on these techniques in isolation. While a few comparative analyses [9] and simple combinations [10] have been proposed, systematic integration of these techniques remains underexplored. However, as models continue to grow in scale, the integration of multiple compression methods becomes increasingly important to maintain performance while reducing model complexity—especially for deployment in mobile and embedded environments.

✉ Kisung Seo
ksseo@skuniv.ac.kr

Woon Chae
codns1201@skuniv.ac.kr

[1] Department of Electronics Engineering, Seokyeong University, Jungneung-Dong 16-1, Sungbuk-Gu, Seoul 02713, Korea

In this paper, we first analyze the characteristics and integration constraints of pruning, quantization, and knowledge distillation. We then propose a harmonic mean-based efficiency score that balances model compression rate and performance degradation. Furthermore, we introduce a self-distillation technique that avoids the use of large external teacher models, and a two-stage distillation process applied during fine-tuning and Quantization-Aware Training (QAT) [11].

Based on these elements, we construct an integrated compression pipeline that respects practical constraints while leveraging the strengths of each method. We evaluate the proposed approach using the ResNet-56 [12], MobileNetV2 [13], model on the CIFAR-100 [14], Tiny ImageNet [15], dataset, conducting various experiments on partial combinations of pruning and quantization. Metrics such as accuracy, FLOPs, and model size are compared. We also extend the evaluation to benchmark the proposed pipeline against existing compression approaches.

2 Related Work

2.1 Pruning

Pruning performs partial structural modifications by reducing the number of nodes or filters within each layer, while keeping the overall block or layer structure of the network fixed. It can reduce model size and improve processing speed; however, performance degradation is common. TAS [16] determines the number of filters in each layer through network architecture search and additionally applies knowledge distillation from a large network. HRank [17] ranks filters based on the feature maps generated from input images and prunes those with low ranks.

2.2 Quantization

Quantization reduces memory and computation by converting weights and activations from 32-bit floats to lower-bit formats like FP16 or INT8, enabling model compression and faster inference. FP16 halves memory usage, while INT8 can achieve up to 4× efficiency gains, supporting near real-time inference on edge devices. Unlike pruning, quantization has fixed bit-width reductions but generally yields greater size and speed improvements. Representative quantization methods are as follows. QNN [11] quantizes weights and activations to binary (± 1) or low-bit (2–4 bit) integers while allowing training. INQ [18] proposes a strategy that progressively quantizes network weights to minimize loss, showing that the original performance can be maintained even in low-bit models.

Table 1 Constraints on the combination of model compression techniques from a single-model perspective

Combination Methods	Feasibility	Remarks
Pruning ->Quantization	○	
Pruning ->KD	△	External Teacher Model is required
Quantization ->Pruning	X	Data Incompatibility
Quantization ->KD	X	Data Incompatibility
KD ->Pruning	△	External Teacher Model is required
KD ->Quantization	△	External Teacher Model is required

2.3 Knowledge Distillation

Knowledge Distillation trains a smaller student network to mimic a larger, pre-trained teacher, transferring knowledge without altering architecture. This enables compression with minimal performance loss, unlike pruning, and requires both teacher and student, with the student's size predefined. The most widely used configuration in knowledge distillation is the Teacher-Student (T-S) framework [8]. In addition, Deep Mutual Learning (DML) [19] is a collaborative model composed of two equal networks that perform mutual distillation. DML allows knowledge to be distilled between identical networks.

3 Constraints in Combining Compression Techniques and Evaluation Metric

3.1 Limitations in Combination

When combining compression techniques, quantization introduces compatibility issues due to changes in data types. While pruning and knowledge distillation are typically performed using FP32 representations, quantization converts model parameters to FP16 or INT8 formats. As a result, quantization can only be applied after pruning or distillation, but not before. Furthermore, knowledge distillation requires an additional large-scale teacher model, which must be provided separately from the target model for pruning or quantization. The compatibility and constraints of each combination are summarized in Table 1.

3.2 Efficiency Evaluation Metrics

To fairly compare and evaluate the performance of model compression techniques, we propose a new evaluation metric that considers the changes in FLOPs, model size, and accuracy. First, the reduction rate is defined in Eq. (1), where X denotes either FLOPs or model size:

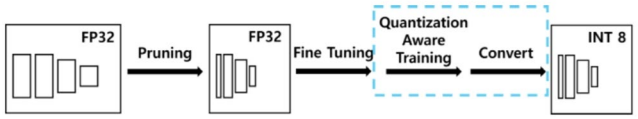


Fig. 1 Combined structure of pruning and quantization

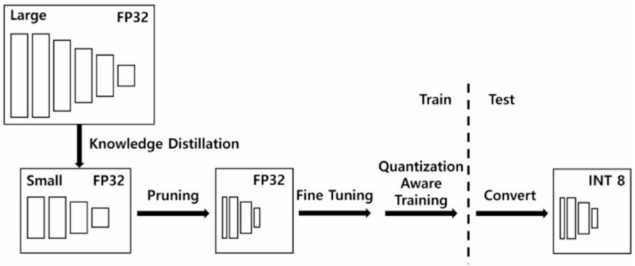


Fig. 2 Integrated pipeline combining pruning, quantization, and Knowledge Distillation (KD)

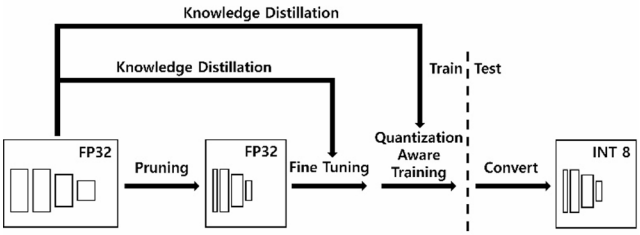


Fig. 3 The proposed pipeline

$$X_{drop} = \frac{X_{full\ precision} - X_{compressed}}{X_{fullprecision}} \tag{1}$$

Based on this, the Compression Score, which reflects only the degree of model compression, is defined as the average of the reduction rates of FLOPs and model size, as shown in Eq. (2). A higher Compression Score indicates that the model has been reduced more significantly in terms of computational cost or size.

$$Compression\ Score = \frac{Flops_{drop} + Model\ Size_{drop}}{2} \tag{2}$$

To assess the trade-off between accuracy and compression, we define an Efficiency Score using the harmonic mean of the Compression Score and Accuracy, as shown in Eq. (3).

$$Efficiency\ Score = \frac{2 \times\ Acc \times\ Compression\ Score}{Acc + Compression\ Score} \tag{3}$$

This metric allows for a quantitative analysis of the trade-off between model size reduction and performance. A higher Efficiency Score indicates better overall efficiency in terms of maintaining accuracy while reducing model complexity.

4 Integrated Lightweight Pipeline

4.1 Combination of Pruning and Quantization

When combining pruning and quantization, pruning must be performed first due to data type changes introduced by quantization [20]. After pruning, fine-tuning is required to recover performance, and a Quantization-Aware Training (QAT) process is necessary during training to simulate the effects of quantization. This ensures that the final model maintains high accuracy even with low-precision formats such as INT8. The structural diagram for the combination of pruning and quantization is shown in Fig. 1.

4.2 Combination of Pruning, Quantization, and Knowledge Distillation

Conventional Knowledge Distillation (KD) is primarily used to transfer knowledge from a large-scale teacher model to a smaller student model in order to improve the performance of the latter. Therefore, pruning and quantization can be applied directly to a given model for model compression without requiring additional components. However, when combining KD with pruning or quantization, an additional teacher model must be provided (refer to Sect. 3.1). The full pipeline that integrates pruning and quantization with such a KD structure is shown in Fig. 2, and we refer to it as the Integrated pipeline, However, since this method requires an additional large teacher model, it demands more computational resources and involves greater effort in managing the KD process.

4.3 Proposed Unified Pipeline

In this study, we propose a Compression-Aware Knowledge Distillation (CAKD) approach, which differs from T-S KD frameworks relying on large external teacher models. In our method, the pre-compression model (before pruning or quantization) serves as the teacher, and the compressed model acts as the student, enabling distillation during both fine-tuning after pruning and quantization-aware training (QAT). The overall structure of the proposed unified pipeline is shown in Fig. 3. This approach reduces the structural gap between the teacher and student models, enabling more stable knowledge transfer. Furthermore, it eliminates the need to train a separate teacher model, as the original pre-trained model can be reused directly, offering significant advantages in terms of computational efficiency. The complete pipeline procedure is summarized in Algorithm 1.

Algorithm 1 Model Compression with Pruning, KD, and QAT

$\theta_T \leftarrow \text{TrainFullPrecisionModel}(D)$
$\theta_P \leftarrow \text{ApplyFilterPruning}(\theta_T, \rho)$
repeat
  for $(x, y) \in D$ **do**
    $z_T \leftarrow f_T(x)$
    $z_P \leftarrow f_P(x; \theta_P)$
    $\mathcal{L} \leftarrow (1-\alpha) \cdot \text{CE}(z_P, y) + \alpha \cdot \text{KD}(z_P, z_T; T)$
    $\theta_P \leftarrow \theta_P - \eta \nabla_\theta \mathcal{L}$
  end for
until converged
$f_{\text{QAT}} \leftarrow \text{PrepareQATModel}(f_P)$
$f_{\text{QAT}} \leftarrow \text{CalibrateObservers}(f_{\text{QAT}}, D)$
$\theta_{\text{QAT}} \leftarrow \theta_P$
repeat
  for $(x, y) \in D$ **do**
    $z_T \leftarrow f_T(x)$
    $z_Q \leftarrow f_{\text{QAT}}(x; \theta_{\text{QAT}})$
    $\mathcal{L} \leftarrow (1-\beta) \cdot \text{CE}(z_Q, y) + \beta \cdot \text{KD}(z_Q, z_T; T)$
    $\theta_{\text{QAT}} \leftarrow \theta_{\text{QAT}} - \eta \nabla_\theta \mathcal{L}$
  end for
until converged
$\theta_Q \leftarrow \text{QuantizeToINT8}(\theta_{\text{QAT}})$
return θ_Q

The first step is to sufficiently train a full-precision model, which will later serve as the teacher model during the compression process. Here, θ_T denotes the parameters of this full-precision teacher model, and $D = \{(x, y\}$ represents the training dataset consisting of inputs and their ground-truth labels. In the second step, filter pruning is applied to remove less important channels, thereby simplifying the model structure. Here, θ_P denotes the parameters of the pruned model obtained after applying filter pruning, and ρ represents the pruning ratio that determines the fraction of channels to be removed. In the third step, fine-tuning is performed to recover the performance loss caused by pruning, during which the logits of the full-precision teacher model are distilled. Here, z_T and z_P denote the logits of the teacher and pruned models, respectively, while $CE(\cdot, \cdot)$ refers to the cross-entropy loss with ground-truth labels and $KD(\cdot, \cdot; T)$ denotes the knowledge distillation loss computed with T temperature. The fourth step involves Quantization-Aware Training (QAT), which simulates integer (INT8) quantization during training. In this phase, knowledge distillation is again applied using the outputs of the full-precision teacher model to enhance the quantization adaptability. In the final step, the QAT-trained model is converted to an integer (INT8) format, resulting in the final compressed model. Here, θ_Q denotes the parameters of the final quantized model used for deployment.

5 Experiments

5.1 Input and Execution Conditions

Pruning is performed on a single network, where the reduction ratio can be adjusted by the user. Quantization is applied to a single network as well, and since it only reduces precision from 32-bit to 16-bit or 8-bit without changing the structure, no separate input for reduction ratio is required. In contrast, knowledge distillation requires a pair of teacher and student networks, and the reduction ratio cannot be arbitrarily adjusted as it is fixed by the size of the student network.

Additionally, pruning typically involves deleting some layers or filters gradually to prevent sudden performance degradation, followed by repeated fine-tuning. Quantization fixes the network structure and only changes numerical precision; in particular, QAT simulates quantization effects during training before converting the model to integer (INT8) format. Knowledge distillation differs in that it operates on two fixed-structure networks for a limited number of iterations. Furthermore, pruning involves structural changes to the network through filter removal until the final result is achieved, whereas knowledge distillation and quantization adjust only parameters without structural modification.

5.2 Experimental Environment

Experiments on the three model compression techniques, their combinations, and the overall integrated pipeline were conducted using the CIFAR-100 and Tiny ImageNet dataset. CIFAR-100 consists of 32×32 RGB images grouped into 20 superclasses such as vehicles, insects, and flowers, with each superclass containing 5 subclasses. The dataset includes a total of 50,000 training images and 10,000 test images. Tiny ImageNet contains 200 classes of 64×64 images with 100,000 training images and 10,000 validation images. The target network used in the experiments were ResNet-56, MobileNetV2 and a larger ResNet-110 model was employed as the common teacher model for comparison with the conventional knowledge distillation method.

The experimental setup is as follows: the implementation was based on the PyTorch framework and executed on NVIDIA RTX 3090 GPUs. For full-precision training, the batch size was set to 128, the learning rate to 0.1, the total number of epochs to 200, and learning rate decay was applied at epochs 100 and 150.

5.3 Pruning Application and Fine-tuning

Filter pruning was applied to the trained baseline model, with pruning ratios set to 0.3, 0.4, and 0.5. After pruning, fine-tuning was conducted to recover performance. The fine-tuning used a batch size of 128, a learning rate of 0.1, 100 epochs, and learning rate decay at epochs 50 and 75, which differ slightly from the initial training settings.

The experimental results for pruning are shown in Table 2. Pruning reduces both the computational cost (FLOPs) and model size simultaneously by shrinking the model structure

Table 2 Comparison results for pruning with fine-tuning on CIFAR100 with ResNet56

Model	Accuracy (%,↑)	FLOPs (M, ↓)	FLOPs drop Ratio (↑)	Model Size (MB, ↓)	Model Size Drop Ratio (↑)
Baseline	71.84	253.70		3.44	
Pruning (0.3)	69.18	120.58	0.52	1.70	0.51
Pruning (0.4)	65.58	87.00	0.66	1.30	0.62
Pruning (0.5)	63.70	32.10	0.87	0.97	0.72

Table 3 Results of quantization-aware training on CIFAR100 with ResNet56

Model	Accuracy (%,↑)	FLOPs (M, ↓)	FLOPs drop Ratio (↑)	Model Size (MB, ↓)	Model Size Drop Ratio (↑)
Baseline	71.84	253.70		3.44	
FP16	71.85	253.70	-	1.78	0.48
INT8-Q	71.09	253.70	-	1.06	0.69

itself. However, in the case of high pruning ratio (0.5), while the FLOPs decrease by approximately 87%, the accuracy drops by 8.14%, indicating a relatively significant performance degradation.

5.4 Quantization-Aware Training and Quantization

Since INT8 quantization involves changing the numerical representation of the model, directly converting a trained model to INT8 may lead to performance degradation. To address this, Quantization-Aware Training (QAT) was employed, allowing the model to learn and compensate for quantization effects during training. The QAT settings were: batch size of 128, learning rate of 0.1, total 20 epochs with learning rate decays at epochs 10 and 15. After QAT, INT8 quantization was applied to convert the model's data representation from floating-point to integer format.

In this study, INT8-Q refers to the model obtained by applying INT8 quantization after the QAT process. Since quantization does not alter the model architecture, the number of floating-point operations (FLOPs) remains unchanged. However, reducing the bit-width of numerical representations results in a smaller model size.

The experimental results for quantization are summarized in Table 3. FP16 quantization achieved a slight accuracy improvement of 0.01% while reducing the model size by approximately 48%. In contrast, INT8 quantization reduced the model size by about 68% but caused a 0.75% drop in accuracy.

5.5 Combined Pruning and Quantization Experiments

The experimental results for the combination of pruning and quantization are presented in Table 3. In some cases, the model compression rate and accuracy degradation rate do not show a consistent pattern across different configurations.

For example, when comparing the 0.5 Pruning with FP16 model and the 0.3 Pruning with INT8 model, the preferred model may vary depending on the evaluation metric. This highlights that neither model size reduction nor performance preservation alone is sufficient for fair comparison; thus, the Efficiency Score defined in Sect. 3.2 is used as a unified evaluation criterion.

Applying FP16 quantization after pruning results in smaller accuracy degradation but offers relatively less model compression. In contrast, INT8 quantization achieves significantly higher compression, albeit with slightly larger accuracy loss. Overall, in terms of the Efficiency Score, the INT8 quantized models achieved slightly better performance, suggesting a more favorable trade-off between compression and accuracy.

5.6 Proposed Integrated Pipeline Experiments

The proposed integrated method applies knowledge distillation (KD) during both the fine-tuning stage after filter pruning and the quantization-aware training (QAT) stage, using the original pre-compression baseline model as the teacher. This approach aims to minimize performance degradation during the sequential application of pruning and quantization. The distillation loss uses a temperature parameter of $T=4$, and the total loss is defined as Eq. (4).

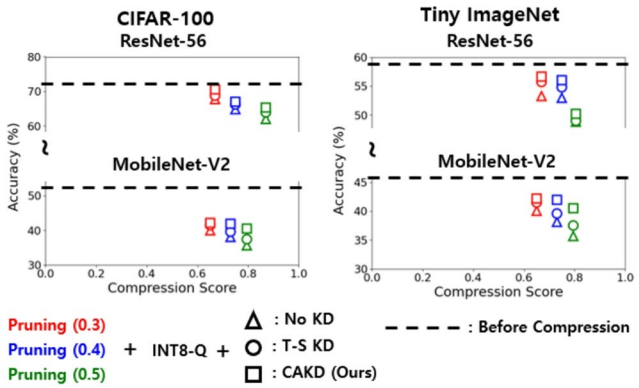


Fig. 4 2-D plots of accuracy versus compression score across methods and models on CIFAR-100 and tiny imageNet (ResNet-56 and MobileNetV2)

$$(1-\alpha)*loss_{ce} + \alpha*loss_{kd},\ where\ \alpha = 0.3 \tag{4}$$

Figure 4 presents performance comparison plots of different methods, illustrating accuracy versus compression score on the CIFAR-100 and Tiny ImageNet datasets using ResNet-56 and MobileNetV2. The figure compares the performance of the proposed CAKD method with T-S KD and the no-KD approach across the top three configurations (based on the Efficiency Score) selected from Table 4. The proposed method outperforms the others across two datasets and two models, demonstrating that using the uncompressed baseline model as a teacher effectively compensates for the accuracy degradation caused by each compression technique.

6 Conclusion

In this study, we analyzed the characteristics and limitations of combining three representative model compression techniques, pruning, quantization, and knowledge distillation, and proposed an integrated pipeline based on Compression-Aware Knowledge Distillation (CAKD). Experiments were conducted on the CIFAR-100 and ImsageNet-200 datasets using ResNet-56 and MobileNetV2 models. Various experiments were carried out on individual pruning, pruning combined with quantization, and the proposed integration of pruning, quantization, and knowledge distillation. The results demonstrate that the proposed CAKD-based pipeline outperforms conventional integration methods in both model compression rate and accuracy. Future work includes extending the evaluation to a wider range of datasets and network architectures.

Acknowledgements This Research was supported by Seokyeong University in 2023.

Author Contributions Woon Chae: Methodology, Software. Kisung Seo: Supervision, Methodology, Writing.

Funding This Research was supported by Seokyeong University in 2023.

Data Availability All data analyzed in this work are from the publicly accessible.

Declarations

Conflict of Interest The authors declare no conflict of interest.

References

1. LeCun Y, Bengio Y, Hinton G (2015) Deep learning. Nature 521:436–444
2. Krizhevsky A, Sutskever I, Hinton GE. (2012) Imagenet classification with deep convolutional neural networks. In: Advances in Neural Information Processing Systems, pp. 1097–1105
3. Moya V, Quito A, Pilco A, Vásconez JP, Vargas C (2024) Crop detection and maturity classification using a YOLOv5-based image analysis. Emerg Sci J 8(2):496–512
4. Goh KW, Surono S, Afiatin MYF, Mahmudah KR, Irsalinda N, Chaimanee M, Onn CW (2024) Comparison of activation functions in convolutional neural network for Poisson noisy image classification. Emerg Sci J 8(2):592–602. https://doi.org/10.28991/ESJ-2024-08-02-014
5. Han S, Mao H, Dally WJ (2015) Deep compression: compressing deep neural networks with pruning, trained quantization and Huffman coding. arXiv:1510.00149
6. Luo JH, Wu J, Lin W (2017) ThiNet: a filter level pruning method for deep neural network compression. In: International Conference on Computer Vision (ICCV), pp. 5068–5076
7. Jacob B et al (2018): Quantization and training of neural networks for efficient integer-arithmetic-only inference. In: IEEE

Table 4 Experimental results of combined pruning and quantization

Model		Accuracy	FLOPs	Model Size	Compression Score (↑)	Efficiency Score (↑)
Pruning	Quantization	(%,↑)	(M, ↓)	(MB, ↓)		
Baseline		71.84	253.70	3.44		
0.3	FP16	69.20	120.58	0.91	0.630	0.660
0.4	FP16	65.57	87.00	0.71	0.725	0.689
0.5	FP16	63.66	64.20	0.55	0.794	0.705
0.3	INT8-Q	67.86	120.58	0.60	0.675	0.677
0.4	INT8-Q	64.84	87.00	0.50	0.756	0.698
0.5	INT8-Q	62.14	64.20	0.42	0.812	0.706

conference on Computer Vision and Pattern Recognition (CVPR), pp. 2704–2713
8. Hinton G, Vinyals O, Dean J (2015) Distilling the knowledge in a neural network. In: advances in Neural Information Processing Systems (NIPS)
9. Minnehan B, Savakis A (2019) Cascaded projection: end-to-end network compression and acceleration. In: IEEE conference on Computer Vision and Pattern Recognition (CVPR), pp. 10715–10724
10. Lee S, Seo K (2020) Hybrid pruning of deep learning system. Trans Korean Inst Electr Eng 69(11):1750–1754
11. Hubara I, Courbariaux M, Soudry D, El-Yaniv R, Bengio Y (2016) Quantized neural networks: training neural networks with low precision weights and activations. arXiv:1609.07061
12. He K, Zhang X, Ren S, Sun J (2016) Deep residual learning for image recognition. In: IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 770–778
13. Sandler M, Howard A, Zhu M, Zhmoginov A, Chen LC (2018) MobileNetV2: Inverted residuals and linear bottlenecks. In: IEEE conference on Computer Vision and Pattern Recognition (CVPR), pp. 4510–4520
14. Krizhevsky A (2009). Learning multiple layers of features from tiny images (Master's thesis). Department of Computer Science, University of Toronto
15. Le, Y.,Yang, X. (2015). Tiny ImageNet visual recognition challenge (Technical Report). Stanford University, CS231n
16. Dong, X, Yang, Y. (2019). Network pruning via transformable architecture search. In Advances in Neural Information Processing Systems (NeurIPS)
17. Lin M, Ji R, Wang Y, Zhang Y, Zhang B, Tian Y, Shao L (2020) In CVPR. Filter pruning using high-rank feature map, HRank
18. Zhou A, Yao A, Guo Y, Xu L, Chen Y (2017) Incremental network quantization: towards lossless CNNs with low-precision weights. ICLR
19. Zhang Y, Xiang T, Hospedales TM, Lu H (2018) Deep mutual learning. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)
20. Yi M, Hwang S (2025) A new lightweight deep learning model optimized with pruning and dynamic quantization to detect freezing gait on wearable devices. Comput Biol Med 191:107159

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.



Woon Chae Received B.S. degrees from Electronics Engineering from Seokyeong University, Seoul, Korea, in 2024. He is currently pursuing his MS degree in Electronics and Computer Engineering at Seokyeong University, His research interests include deep learning, computer vision.



Kisung Seo Received the BS, MS, and Ph.D degrees in Electrical Engineering from Yonsei University, Seoul, Korea, in 1986, 1988, and 1993 respectively. He became Full Time Lecturer and Assistant Professor of Industrial Engineering in 1993 and 1995 at Seokyeong University, Seoul, Korea. He joined Genetic Algorithms Research and Applications Group (GARAGe) and Case Center for Computer-Aided Engineering & Manufacturing, Michigan State University from 1999 to 2002 as a Research Associate. He was also appointed Visiting Assistant Professor in Electrical & Computer Engineering, Michigan State University from 2002 to 2003. He was a Visiting Scholar at BEACON (Bio/computational Evolution in Action CONsortium) Center, Michigan State University from 2011 to 2012. He is currently Professor of Electronics Engineering, Seokyeong University. His research interests include deep learning, evolutionary computation, fuzzy-neural networks, computer vision, intelligent robotics.